

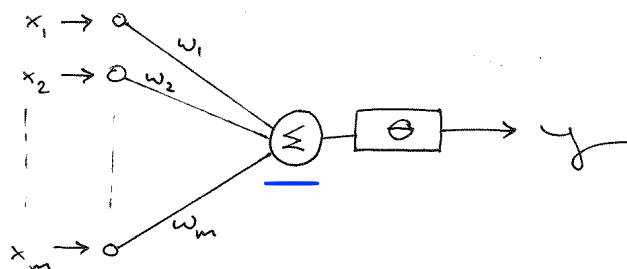
# COMS5026A

## Applied Machine Learning

### 1. Perceptrons

#### 1.1. Artificial Neurons.

An **artificial neuron** has the following structure:



The values  $w_1, \dots, w_m$  are real numbers called the **weights**.

The value of  $\theta$  is a real number called the **threshold**.

Given input values  $x_1, \dots, x_m$ , where each  $x_i \in \mathbb{R}$ , the output  $y$  is obtained as follows:

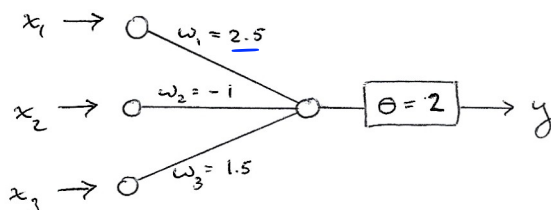
First calculate the sum: 
$$h = \sum_{i=1}^m w_i x_i.$$

Then, compare the sum  $h$  with the **threshold** value  $\theta$ :

if  $h > \theta$  then the output value is  $y = 1$ ; (we say the neuron **fires** or **activates**)

if  $h \leq \theta$  then the output value is  $y = 0$ .

**Example:** Consider the following 3-input artificial neuron:



Suppose the following values are input to neuron:  $x_1 = 1$ ,  $x_2 = 3$ ,  $x_3 = 2$ , i.e., the input vector is  $\mathbf{x} = (1, 3, 2)$ . The corresponding output is obtained as follows:

$$\sum_{i=1}^3 w_i x_i = (2.5)1 + (-1)3 + (1.5)2 = 2.5 \quad \text{and} \quad 2.5 > 2, \quad \text{so } y = 1.$$

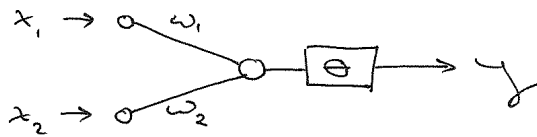
If the input vector is  $\mathbf{x} = (1.5, 1, -1)$  the corresponding output is obtained as follows:

$$\sum_{i=1}^3 w_i x_i = (2.5)(1.5) + (-1)1 + (1.5)(-1) = 1.25 \quad \text{and} \quad 1.25 \leq 2, \quad \text{so } y = 0.$$

A function made up of a single artificial neuron is sometimes called a perceptron.

## 1.2. Perceptrons as linear discriminants.

Consider a 2-input perceptron, which is defined by the weights  $w_1$  and  $w_2$  on the input edges, together with the threshold value  $\theta$ :



For input  $\mathbf{x} = (x_1, x_2)$  the output  $y$  is calculated as follows:

if  $w_1 x_1 + w_2 x_2 > \theta$ , then  $y = 1$ ;

if  $w_1 x_1 + w_2 x_2 \leq \theta$ , then  $y = 0$ .

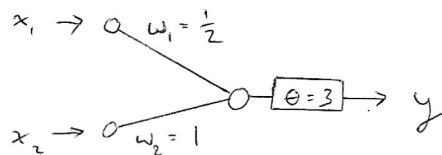
Thus, the equation  $w_1 x_1 + w_2 x_2 = \theta$  (where  $w_1$ ,  $w_2$  and  $\theta$  are constants and  $x_1$  and  $x_2$  are the variables) separates inputs with output  $y = 1$  from those with output  $y = 0$ .

The equation  $w_1 x_1 + w_2 x_2 = \theta$  defines a straight line in  $\mathbb{R}^2$ .

The equation can also be written as  $w_1 x_1 + w_2 x_2 - \theta = 0$ .

The line defined by the equation is called a linear discriminant.

As an example, consider the 2-input perceptron defined below:

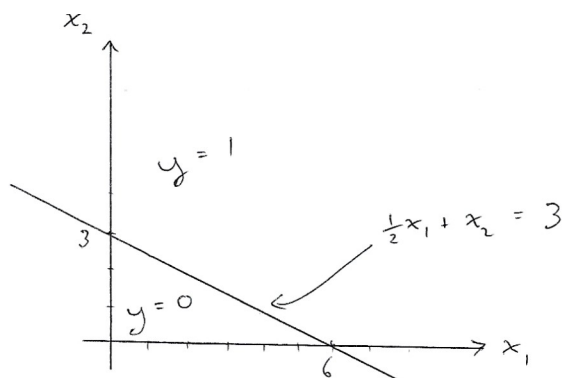


The linear discriminant defined by this perceptron is  $\frac{1}{2}x_1 + x_2 = 3$ , or  $\frac{1}{2}x_1 + x_2 - 3 = 0$ .

A sketch of the line is given below; all points  $(x_1, x_2)$  below the line will produce output  $y = 0$ , and all points  $(x_1, x_2)$  above the line will produce output  $y = 1$ .

Thus, the line  $\frac{1}{2}x_1 + x_2 = 3$  separates  $\mathbb{R}^2$  into two regions corresponding to the output of the perceptron.

As an exercise, try a few points from below the line, say  $(1, 1)$ ,  $(0, 5)$ ,  $(-4, -1)$ , and check that the output is 0; and try a few points from above the line, say  $(1, 4)$ ,  $(2, 3)$ ,  $(10, 10)$ , and check that the output is 1.

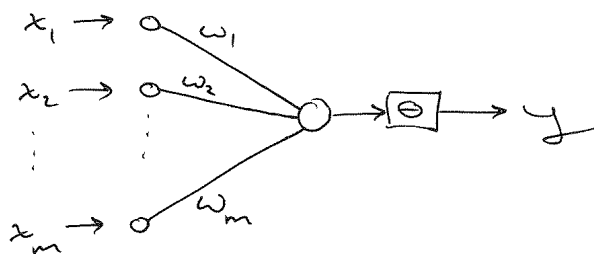


A 3-input perceptron, similarly, has an associated equation that defines a plane  $\mathbb{R}^3$ .

A plane is a linear discriminant that separates  $\mathbb{R}^3$  into two regions.

In general, an  $m$ -input perceptron defines a linear discriminant in  $\mathbb{R}^m$ , or an  $m$ -**hyperplane**.

Consider a neuron with  $m$  inputs as below.



The neuron fires, i.e.,  $y = 1$ , if  $\sum_{i=1}^m w_i x_i > \theta$ ,

and does not fire, i.e.,  $y = 0$ , if  $\sum_{i=1}^m w_i x_i \leq \theta$ .

The linear discriminant associated with this perceptron is  $\sum_{i=1}^m w_i x_i = \theta$ .

### 1.3. Applying a Perceptron to a dataset.

Next, we show how a perceptron can be applied to a dataset.

As an example, consider the following small dataset:

$$D = \{((3, 1), 1), ((2, 2.5), 0), ((2, 1.5), 1), ((4, 3), 1), ((3, 3), 0)\},$$

which consists of a set of points in  $\mathbb{R}^2$ , each with a target class of either 1 or 0.

For example, the point  $(3, 1)$  has class 1; the point  $(2, 2.5)$  has class 0.

The dataset can also be represent by the following tables:

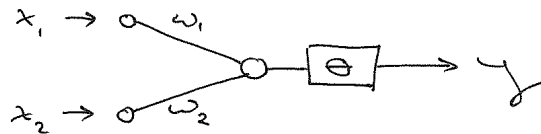
$$X = \begin{bmatrix} 3 & 1 \\ 2 & 2.5 \\ 2 & 1.5 \\ 4 & 3 \\ 3 & 3 \end{bmatrix} \quad T = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}.$$

The rows of  $X$  contain the input values and values in  $T$  are the corresponding classifications.

The goal is to find a function that correctly classifies each input in the dataset.

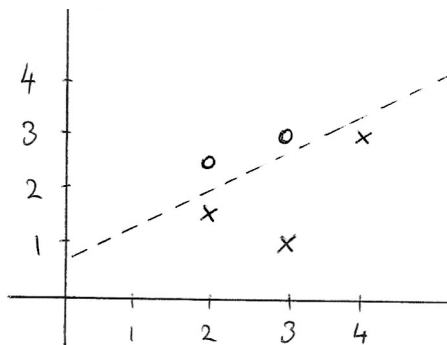
This is an example of a **classification** problem.

A possible solution to the above classification problem is to use a 2-input perceptron:



We want to find values for  $w_1$ ,  $w_2$  and  $\theta$  such that for each pair of values in the set  $X$ , say  $(x_1, x_2)$ , if we feed these values into the perceptron, then the output  $y$  will be the corresponding target value in  $T$ .

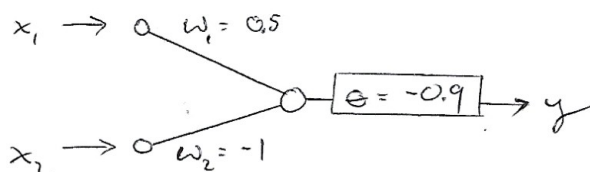
In the diagram below the points of the dataset have been plotted with their classes; the  $\times$ 's indicate points classified as 1 and the  $\circ$ 's indicate points classified as 0.



In the diagram, a straight line that serves as a linear discriminant for the dataset is shown.

The equation for the straight line is  $0.5x_1 - x_2 + 0.9 = 0$ , or  $0.5x_1 + (-1)x_2 = -0.9$ .

Thus, we can define a perceptron that has this line as its linear discriminant by setting  $w_1 = 0.5$ ,  $w_2 = -1$  and  $\theta = -0.9$ , as shown.



We can check the following results:

For input  $(x_1, x_2) = (3, 1)$ , the output is obtained as follows:

calculate  $w_1x_1 + w_2x_2 = (0.5)3 + (-1)(1) = 0.5$ ,

then compare the result with  $\theta$ : since  $0.5 > -0.9$  we get  $y = 1$ , which is correct.

For input  $(x_1, x_2) = (2, 2.5)$ , the output is obtained as follows:

calculate  $w_1x_1 + w_2x_2 = (0.5)2 + (-1)(2.5) = -1.5$ ,

then compare the result with  $\theta$ : since  $-1.5 \leq -0.9$  we get  $y = 0$ , which is correct.

As an exercise, you can check the remaining cases in  $X$ .

Now that a suitable perceptron has been found for the dataset above, we can use the perceptron to predict the classification of other inputs. For example, we would predict that the input  $(3, 2)$  should be classified as a 1.

### 1.4. Training a Perceptron.

Suppose we are given a dataset  $D$  containing  $N$  data points of the form  $(\mathbf{x}, t)$ , where each  $\mathbf{x}$  is in  $\mathbb{R}^m$ , and each  $t$  is either 1 or 0. As in the above example, the goal is to solve the classification problem by finding a function that correctly classifies each input in the dataset.

In the above example, there were only a few points and we could sketch these points because they were in  $\mathbb{R}^2$ . Therefore, simply by inspection, it was quite easy to find a straight line separating the classes, and that line gave us the resulting perceptron.

In general, if the dataset is large and if the points are in  $\mathbb{R}^m$  for  $m \geq 2$ , it is very difficult to find a linear discriminant by inspection. In this case, we make use of an algorithm to find a suitable linear discriminant, i.e., a suitable perceptron. The algorithm, called the PERCEPTRON TRAINING ALGORITHM, is given below. It is an example of a machine learning algorithm.

The algorithm starts by randomly choosing values for  $w_1, \dots, w_m$  and  $\theta$ , which defines the initial perceptron. Then, a data point  $\mathbf{x}$  is input into the perceptron and an output  $y$  is obtained, where  $y = 1$  or  $y = 0$ . If the output is the same as the target value for  $\mathbf{x}$ , then there is no change to the perceptron weights. But if the output and target values are different, then the weights  $w_1, \dots, w_m$  and threshold  $\theta$  are changed slightly so that if the same  $\mathbf{x}$  is input again, the output would match the target.

#### PERCEPTRON TRAINING ALGORITHM

Create an  $m$ -input perceptron by randomly choosing weights  $w_1, \dots, w_m$  and randomly choosing a threshold value  $\theta$ .

(Usually the random values are chosen from a fixed range such as  $[-1, 1]$ .)

Choose a **learning rate**  $\eta$ , where  $0 < \eta < 1$ .

(Usually, we choose  $\eta$  between 0.001 and 0.1.)

**while** (stopping condition is not satisfied)

**for** each data point  $(\mathbf{x}, t)$  in the set  $D$ , do the following:

        feed  $\mathbf{x}$  into the perceptron, to get output  $y$

        update each weight  $w_i$  using the rule:  $w_i \leftarrow w_i + \eta(t - y)x_i$

        update the threshold  $\theta$  using the rule:  $\theta \leftarrow \theta - \eta(t - y)$

The number  $\eta$  is called the **learning rate**. It is usually chosen as a small value so that the changes to the weights are made incrementally. Each data point in the dataset will be fed into the perceptron a number of times before convergence is achieved.

One complete execution of the above **for** loop is called an **epoch**.

During an epoch, the order in which the data points are input for learning can make a difference.

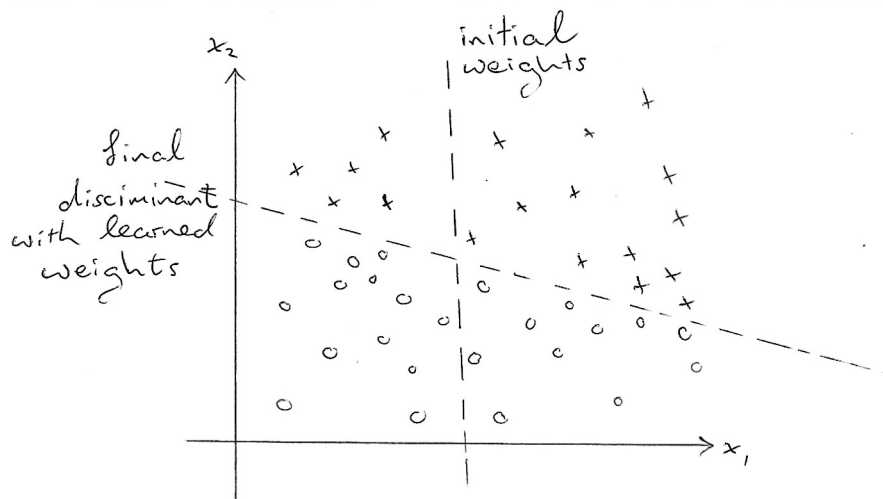
For example, the last data points have more impact than the first ones.

For this reason the data is randomly ordered at the start of each epoch.

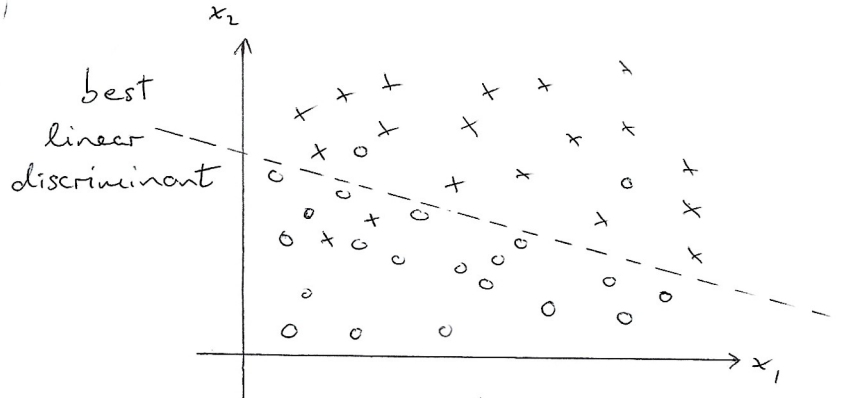
The final weights obtained by the PERCEPTRON TRAINING ALGORITHM give us the trained perceptron. It should be the case that for ‘most of’ the training data, the output from the perceptron matches the target output, i.e., if  $(\mathbf{x}, t)$  is in the dataset and  $\mathbf{x}$  is input into the perceptron, then the output  $y$  should be the same as the target  $t$ .

The diagram below shows plots of the data points in a dataset consisting of points of the form  $(\mathbf{x}, t)$ , where  $\mathbf{x} = (x_1, x_2)$  is in  $\mathbb{R}^2$  and  $t$  is either 1 or 0. Each  $\times$  indicates a point classified as 1, and each  $\circ$  indicates a point classified as 0.

The initial randomly chosen weights of the perceptron define a straight line as shown; the PERCEPTRON TRAINING ALGORITHM is then applied until the weights are adjusted to obtain a perceptron with the final discriminant shown.



It may be that it is impossible to separate the 1's and 0's in the given dataset. In this case, the PERCEPTRON TRAINING ALGORITHM will find a best possible solution that misclassifies as few points as possible. The situation is illustrated in the following diagram.



### 1.5. Stopping Conditions.

We can stop the algorithm if there were no changes to any weights during an epoch. Alternately, since this may not happen, we can stop after some maximum number of epochs or after some specified time has elapsed.

Another way to determine if the perceptron has trained for long enough is to calculate the **loss** (or **error**) of the perceptron on the dataset.

Suppose we have a data set  $D = \{(\mathbf{x}_k, t_k) \mid 1 \leq k \leq N\}$ , where each  $\mathbf{x}_k \in \mathbb{R}^m$  and each  $t_k \in \{0, 1\}$ , and we have trained a perceptron on this dataset.

For each data point  $(\mathbf{x}_k, t_k)$  in  $D$ , feed the input  $\mathbf{x}_k$  into the perceptron to get the output  $y_k$  and calculate the loss  $L$  as follows:

$$L = \sum_{k=1}^N |t_k - y_k|.$$

The above sum counts the number of times the perceptron misclassifies an input.

If  $L$  gets to 0, then the perceptron gives the correct output for every input so we can stop training. Since this may not happen, we may need to stop when  $L$  gets below some fixed value. Since even this is not guaranteed, one can stop when the change in the error from one epoch to the next is close to 0. This means that the perceptron has stopped learning.

The values of  $L$  after every epoch can be plotted in a graph to see how it changes with learning.

*EXERCISES*

- (1) Consider a 3-input perceptron with weights  $w_1 = 2.5$ ,  $w_2 = -3$ ,  $w_3 = 1.5$  and threshold  $\theta = 2$ .

Find the output of the perceptron for the following input vectors:

- (a)  $\mathbf{x} = (-1, 2, 4)$
  - (b)  $\mathbf{x} = (2, -1, -2)$
- (2) Using the perceptron in exercise (1), do one update of the weights  $w_1, w_2, w_3$  and threshold  $\theta$  as follows: Suppose the input vector  $\mathbf{x} = (1, 1, 2)$  from your dataset has target  $t = 0$ . When  $\mathbf{x}$  is fed into the perceptron the output we get is  $y = 1$ , which differs from the target  $t$ . Update all the weights and threshold using the rules in the PERCEPTRON TRAINING ALGORITHM. Use  $\eta = 0.1$ .